Software Architecture and Design Specification (SAD)

Project: Helping Hands (HH), an app that lets management staff of a retail chain volunteer to work in stores during very busy times such as weekends and holidays

Version: 1.0

Authors: Vivek Varma (PES1UG24AM070), Bhuvan Kumar P (PES1UG24AM069), Dhanush Shankar Reddy (PES1UG24AM083), Aamer Khan (PES1UG24AM902)

Date: 28-10-2026

Status: Final

Revision history

Version	Date	Author	Change summary	Approval
1.0	28-10-2026	Vivek Varma	Architecture, class model, sequence diagrams and API design	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Project Guide			

1. Introduction

1.1 Purpose

This document specifies the architecture and design of Helping Hands: the components and how they are layered, the reasons for that structure, the domain model, the main runtime flows, the API and the security design. It turns the requirements in the SRS into a design the team can build within two sprints.

1.2 Scope

It covers the web app, the REST API, authentication and access control, the business services (stores and busy periods, shifts, sign-ups, attendance, notifications, reporting and audit), the data layer and their interfaces. Integration with HR or rostering systems, email and SMS, and calendar export are future work.

1.3 Audience

Developers, QA engineers, the project guide and the course evaluators.

1.4 Definitions

Term	Meaning
Shift	A store's help request for a number of volunteers, for one task, on one date and time.
Sign-up	A volunteer's request to work a shift: Requested, Confirmed, Declined or Withdrawn.
Busy period	A named date range of heavy footfall, such as a holiday weekend.
RBAC	Role-based access control.
Store scope	The rule that a store manager can act only on their own store's shifts and sign-ups.
STRIDE	Threat categories: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege.

2. Document overview

2.1 How to use this document

Section 3 describes the architecture: the component and class diagrams, what each component does, the chosen pattern and why, the technology stack, risks, traceability to the SRS and the security architecture. Section 4 describes the design: the two key runtime flows as sequence diagrams, the API, error handling and UX.

2.2 Related documents

Helping Hands SRS v1.0, Helping Hands Software Test Plan (STP) v1.0, and the RTM in section 8 of the SRS.

3. Architecture

3.1 Goals & constraints

- **Goals:** a shift is never overbooked, even when two managers approve at once; the open-shift list loads within 2 seconds; the sign-up rules can be tested without the UI; and the system can be built and demonstrated by four people in two sprints.
- **Constraints:** web only, used on staff's own phones and laptops; one retail chain; no HR, payroll or messaging integration in this version; volunteered hours are not linked to pay.

3.2 Stakeholders & concerns

Stakeholder	Concern
Management volunteers	Finding a suitable shift fast on a phone, and knowing at once whether a sign-up went through.
Store managers	Enough help for each busy period, choosing volunteers, recording who turned up.
Administrator (operations or HR)	Correct access rights, sensible volunteering limits, reports and an audit trail.
Developers	Clear module boundaries, testable rules, a simple deployment.
Evaluators	Traceability from requirements to design, tests and the working product.

3.3 Component (UML) diagram

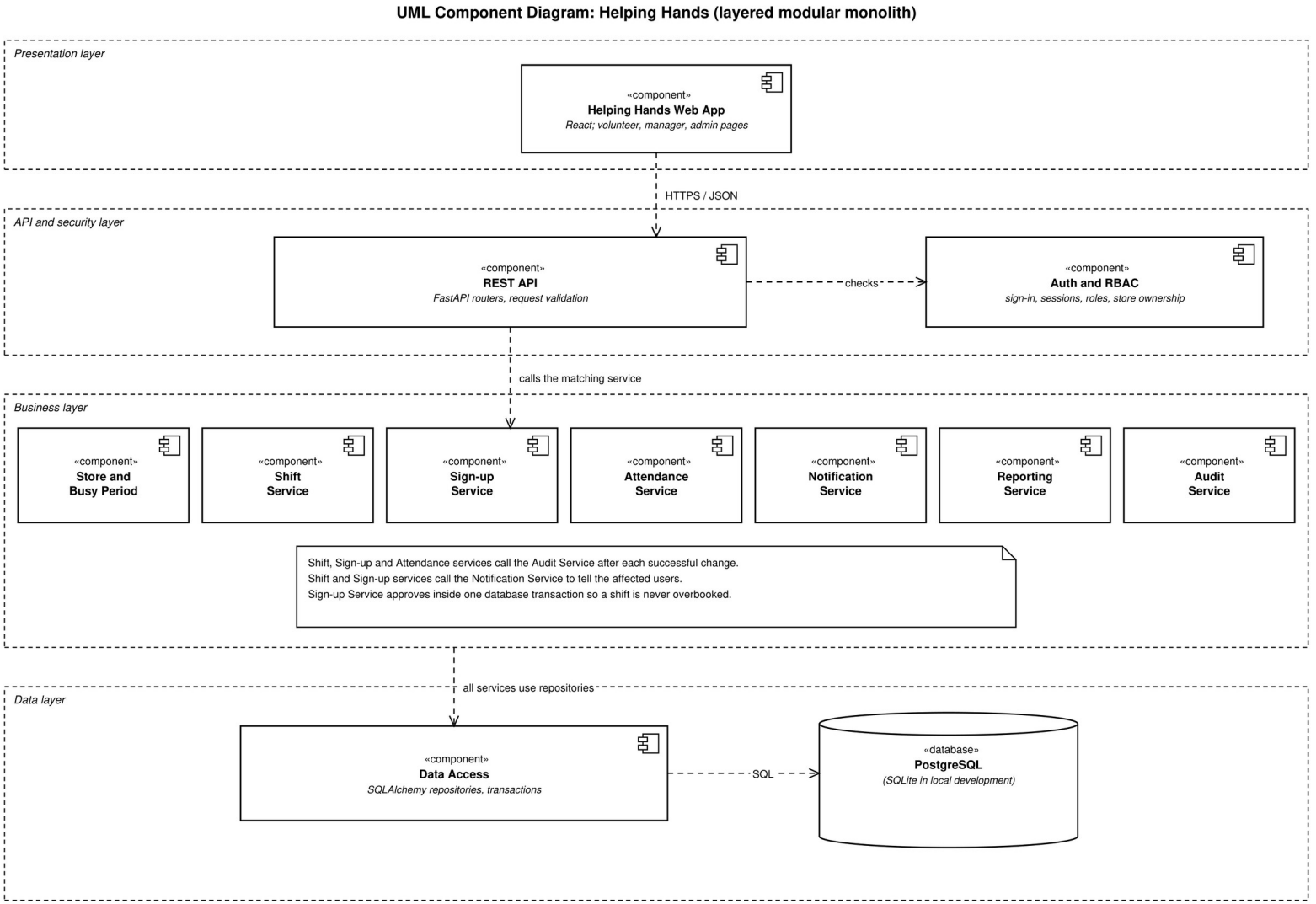


Figure 1. UML component diagram (layered architecture)

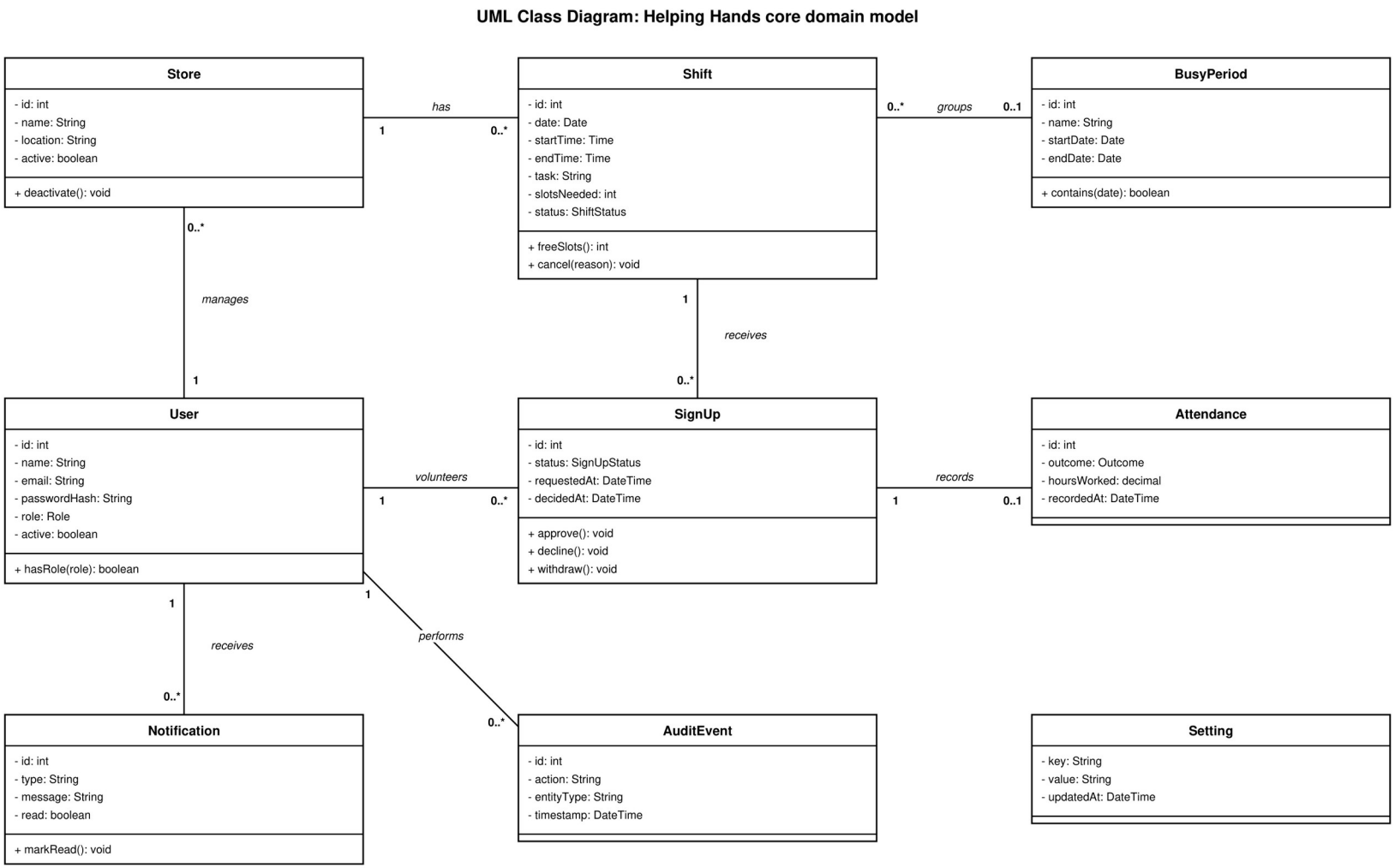


Figure 2. UML class diagram of the core domain model

3.4 Component descriptions

Component	Responsibility	Key interface
Helping Hands Web App	React pages for volunteers, store managers and administrators; forms, filters and status messages; works on phones.	HTTPS / JSON
REST API	FastAPI routers: request validation, calls to the right service, safe error responses.	/api/v1
Auth and RBAC	Sign-in, password hashing, session tokens, lockout, role and store-scope checks.	IAuth
Store and Busy Period	Stores, manager assignment, busy periods and the volunteering rules (limit, notice, cutoff).	IStores
Shift Service	Create, edit, cancel and list shifts; keep status Open, Filled, Cancelled or Completed.	IShifts
Sign-up Service	Eligibility checks, approvals, declines and withdrawals; approval runs in one transaction.	ISignUps
Attendance Service	Attended or no-show and hours worked for each confirmed sign-up.	IAttendance
Notification Service	In-app notifications and the 24-hour reminder job.	INotify
Reporting Service	Coverage per busy period, hours by volunteer and store, no-shows, CSV export.	IReports
Audit Service	Append-only log of who changed what and when.	IAudit
Data Access	SQLAlchemy repositories; every query and transaction goes through here.	SQL
PostgreSQL	All application data (SQLite in local development).	SQL

3.5 Chosen architecture pattern and rationale

A layered architecture (presentation, API and security, business services, data), deployed as one modular application.

Layering means every request passes through the same security layer, so role and store checks are never skipped, and the business rules sit in services that can be unit-tested without the API or the UI. One deployable unit suits a four-person team that has to build, test and demo the system within two sprints. The services are still separate modules, so any of them can be split out later.

Microservices were considered and rejected. They would add service discovery, network calls and distributed data, and would make the no-overbooking rule much harder to guarantee. Instead, the Sign-up Service locks the shift row, counts the confirmed sign-ups and confirms only if a slot is free, all in one database transaction (HH-NF-002).

3.6 Technology stack & data stores

Layer	Technology
Frontend	React 18, Vite, React Router
Backend	Python 3.12, FastAPI, Pydantic, SQLAlchemy 2, Alembic migrations
Data store	PostgreSQL 16 (SQLite for local development)
Security	bcrypt password hashing, signed session tokens (JWT) with 30-minute idle expiry, rate limiting
Scheduling	APScheduler job for the 24-hour reminders

Layer	Technology
Testing	pytest, Vitest, Playwright, JMeter, OWASP ZAP baseline scan
DevOps	GitHub Actions CI, Docker for the test environment

3.7 Risks & mitigations

Risk	Mitigation
Two managers approve the last slot at the same moment	Row lock and transaction in the Sign-up Service, plus an automated concurrency test.
Mistakes with times near the cutoff or across midnight	Store times in the store's local time; test exactly at the cutoff and with back-to-back shifts.
Too few volunteers for a busy period	Coverage view and reminders let managers spot gaps early and repost.
Request floods when a big busy period opens	Rate limiting and an indexed shift list query.
Merge conflicts among four contributors	One module owner each, feature branches, pull requests and CI on every merge.
Data loss on the deployed server	Daily database backup and a tested restore.

3.8 Traceability to requirements

Requirement	Component
HH-F-001 to HH-F-003, HH-SR-002 to HH-SR-004	Auth and RBAC
HH-F-004 to HH-F-006	Store and Busy Period
HH-F-007 to HH-F-010, HH-F-015	Shift Service
HH-F-011 to HH-F-014, HH-F-016, HH-F-019, HH-NF-002	Sign-up Service (transactional approval)
HH-F-017, HH-F-018	Notification Service
HH-F-020	Attendance Service
HH-F-021, HH-F-022	Reporting Service
HH-F-023, HH-SR-007	Audit Service
HH-NF-001, HH-SR-005, HH-SR-008	REST API and indexed queries
HH-NF-004, HH-NF-005	Helping Hands Web App

3.9 Security architecture

Threat modelling (STRIDE) for the main assets: staff accounts, contact details, sign-up decisions and recorded hours.

Threat	Example in Helping Hands	Mitigation
Spoofing	Stolen password used to approve sign-ups	bcrypt hashes, lockout after 5 failures, 30-minute session expiry (HH-SR-002, HH-SR-003)
Tampering	Editing slot counts or hours to change a report	Server-side validation, store-scope checks, append-only audit log (HH-SR-004, HH-SR-007)
Repudiation	A manager denies declining a volunteer	Audit entry with user, time and decision
Information disclosure	A volunteer looks up other staff's phone numbers	Contact details shown only to the manager of a confirmed shift (HH-SR-006); TLS (HH-SR-001)

Threat	Example in Helping Hands	Mitigation
Denial of service	Hundreds of sign-up requests the moment a holiday is posted	Rate limiting (HH-SR-008), indexed queries
Elevation of privilege	A volunteer calls a store manager endpoint	Role checks on every route, deny by default

4. Design

4.1 Design overview

Each module follows router, then service, then repository. Routers handle HTTP and validation, services hold the rules (eligibility, approval, cutoffs, hours), and repositories talk to the database. After a successful change, services call the Audit Service and, where users need to know, the Notification Service. The reminder job runs inside the same application.

4.2 UML sequence diagrams

Sequence diagram 1 shows a volunteer signing up: the eligibility checks run before anything is saved. Sequence diagram 2 shows a store manager approving a sign-up: the slot check and the confirmation happen in one transaction.

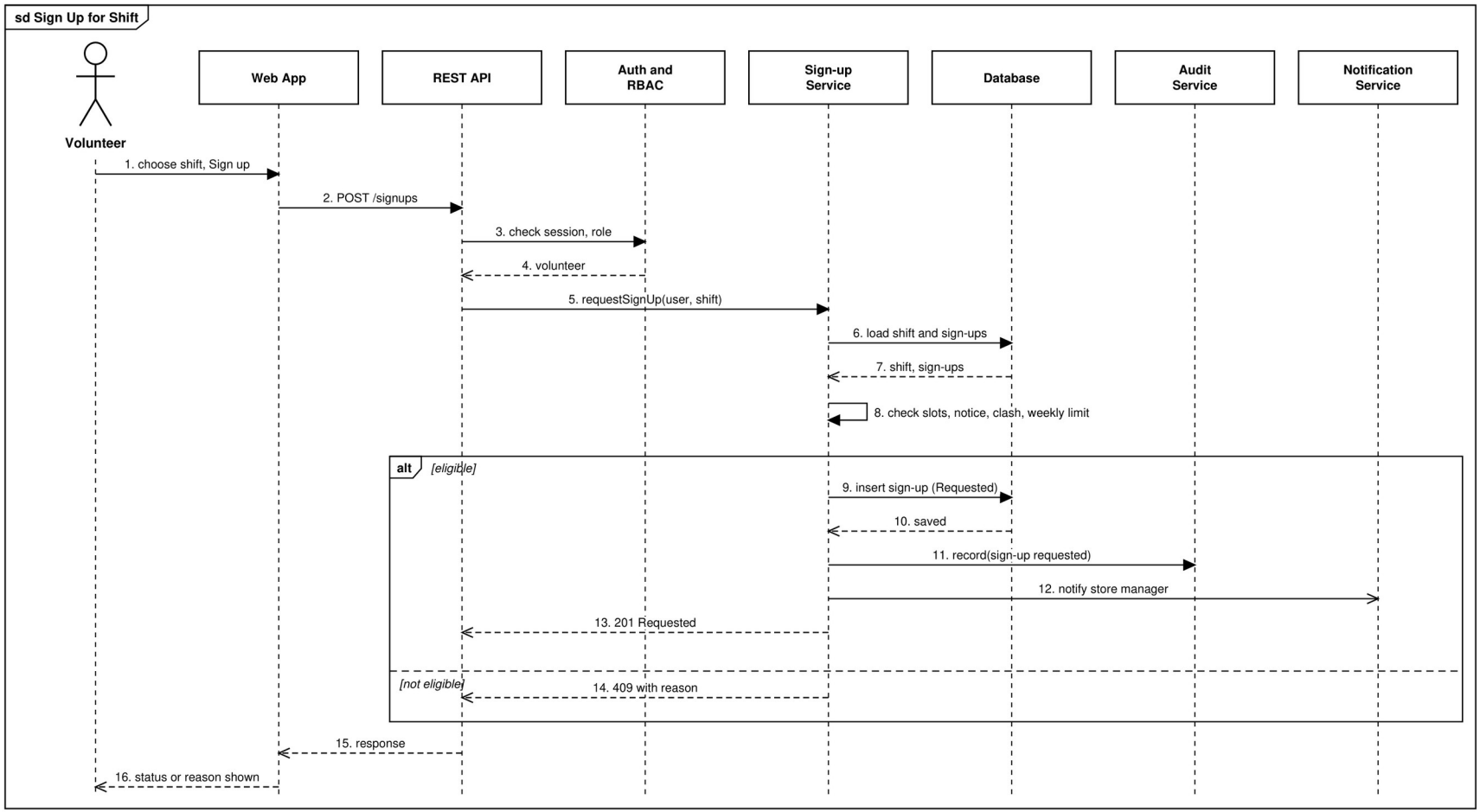


Figure 3. Sequence diagram 1: volunteer signs up for a shift

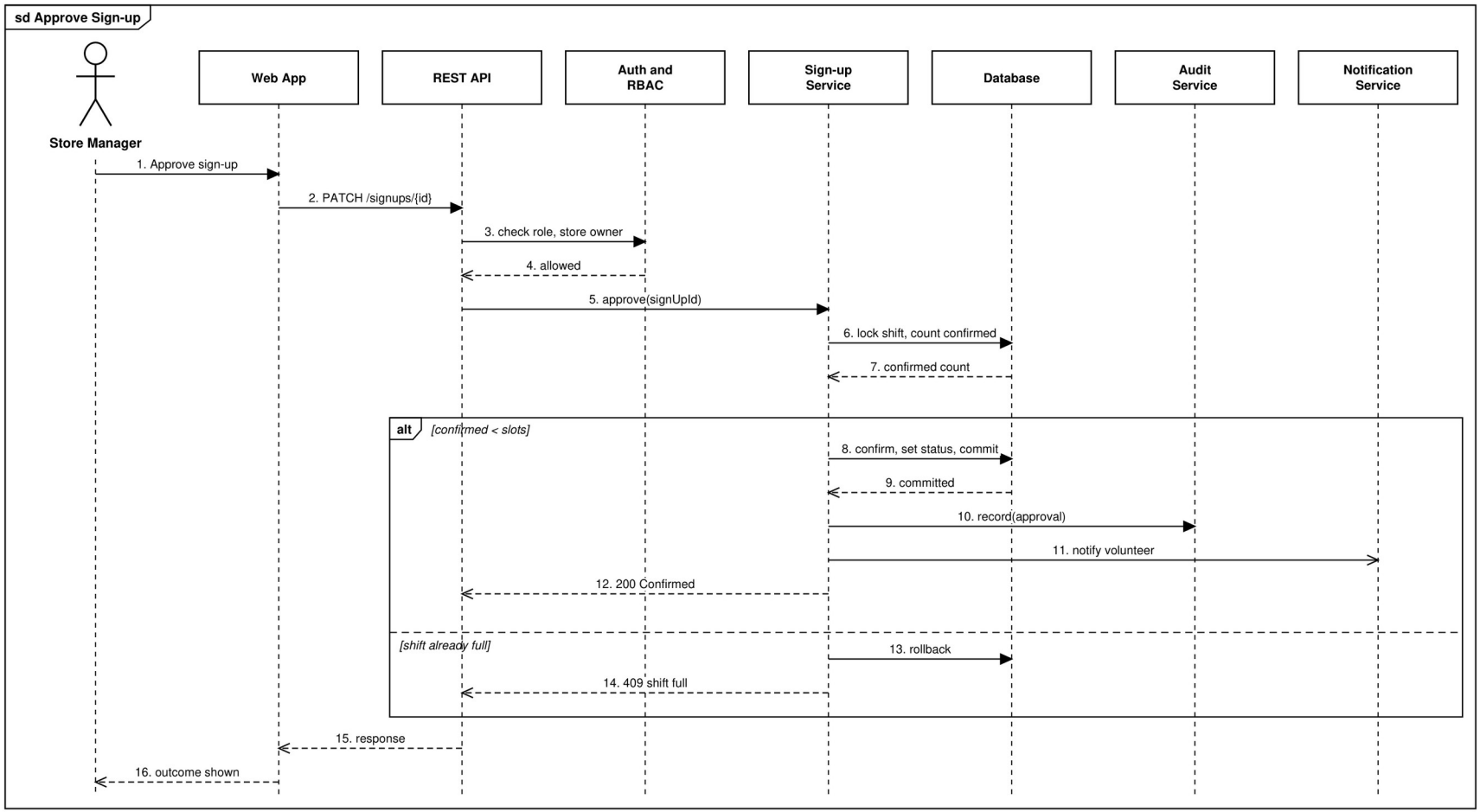


Figure 4. Sequence diagram 2: store manager approves a sign-up

4.3 API design

All endpoints are under /api/v1, use JSON, and need 'Authorization: Bearer <token>'.

Shift Service: post help request

Field	Value
Endpoint	/api/v1/shifts
Method	POST
Allowed roles	STORE_MANAGER (own store only)
Request	{ "storeId": 12, "busyPeriodId": 3, "date": "2026-11-08", "startTime": "10:00", "endTime": "14:00", "task": "Billing", "slotsNeeded": 3, "notes": "Festive rush" }
Response	201 Created { "shiftId": 845, "status": "OPEN", "freeSlots": 3 }
Errors	400 invalid times or slots; 401 not signed in; 403 not this store's manager

Shift Service: list open shifts

Field	Value
Endpoint	/api/v1/shifts?status=OPEN&busyPeriodId=3&storeId=12&task=Billing
Method	GET
Allowed roles	VOLUNTEER, STORE_MANAGER, ADMIN
Request	query parameters only
Response	200 OK [{ "shiftId": 845, "store": "Indiranagar", "date": "2026-11-08", "startTime": "10:00", "endTime": "14:00", "task": "Billing", "freeSlots": 2 }]
Errors	400 invalid filter; 401 not signed in

Sign-up Service: sign up for shift

Field	Value
Endpoint	/api/v1/shifts/{shiftId}/signups
Method	POST
Allowed roles	VOLUNTEER
Request	no body; the volunteer comes from the token
Response	201 Created { "signUpId": 3310, "status": "REQUESTED" }
Errors	409 { "code": "SHIFT_FULL" | "CLASH" | "NOTICE" | "WEEKLY_LIMIT", "message": "..." }; 404 unknown shift; 429 too many requests

Sign-up Service: approve or decline sign-up

Field	Value
Endpoint	/api/v1/signups/{signUpId}
Method	PATCH
Allowed roles	STORE_MANAGER (own store only)
Request	{ "decision": "APPROVE" } or { "decision": "DECLINE" }
Response	200 OK { "signUpId": 3310, "status": "CONFIRMED", "shiftStatus": "FILLED" }
Errors	409 shift already full; 403 another store's shift; 404 unknown sign-up

4.4 Error handling, logging & monitoring

- Errors use one JSON shape, { code, message }, and never show stack traces to users.
- A refused sign-up always says why: full, clash, notice period or weekly limit.
- Unexpected failures are logged with a reference ID; passwords, tokens and contact details are never logged.
- The audit log is kept separate from diagnostic logs.
- A health endpoint (/api/v1/health) and CI test results show the state of the build; failed reminders are logged and retried.

4.5 UX design

- Phone first for volunteers: shift cards with store, date, time, task and free slots, and a single Sign up button.
- Store manager dashboard shows coverage per busy period at a glance, with pending sign-ups at the top.
- Statuses are words (Requested, Confirmed, Full), not colours alone; forms work with the keyboard (WCAG 2.1 AA).
- Before a sign-up, the volunteer sees the withdrawal cutoff and their remaining weekly hours.

4.6 Open issues & next steps

- Whether volunteered hours should count towards recognition or appraisal, which affects how reports are shared.
- Whether email or SMS reminders are needed on top of in-app notifications.
- Whether a store needs more than one manager account.
- Whether the staff list should come from the HR system instead of being entered by the administrator.

5. Appendices

5.1 Glossary: shift, sign-up, slot, busy period, coverage, withdrawal cutoff, minimum notice, weekly hour limit, RBAC, store scope, STRIDE.

5.2 References: Helping Hands SRS v1.0; ISO/IEC/IEEE 42010 (architecture description); OWASP Top 10; NIST SP 800-160.

5.3 Tools: Python (PyMuPDF) for the UML diagrams, Swagger UI generated by FastAPI for the API, Jira for the backlog and sprints.